AhamX Bodhi

Where I Becomes Infinite

Tokenization in Production

Module 3.3: Tokenization — AI Applications

Prof. Sashikumaar Ganesan | IISc Bengaluru

Session Overview

What You Will Learn

- How tokenization affects API cost and latency
- Context window budgeting with tokens
- Tokenization in RAG and agent pipelines
- Tokenizer-aware prompt engineering
- Monitoring tokenization in production systems

Concept at a Glance

- **Duration:** 9 minutes
- **Bloom's Level:** B3 (Apply)
- **Difficulty:** L2 (Intermediate)
- **Domain:** AI Applications
- **Focus:** Production systems and cost optimization

Learning Objectives

By the End of This Concept You Will Be Able To

- **Calculate** the token cost of a prompt and estimate API spend
- **Design** a context budget strategy for RAG and multi-turn conversations
- **Identify** tokenization failure modes in production: cost spikes, context overflow, output artifacts
- **Apply** tokenizer-aware prompt compression techniques
- **Monitor** token usage with LLM observability tools

The Token Economy

Every Token Has a Price

- All commercial LLM APIs charge by the token for both input and output
- **GPT-4o (2025):** $2.50 per million input tokens, $10 per million output tokens
- **Claude 3.7 Sonnet (2025):** $3 per million input, $15 per million output tokens
- **Gemini 1.5 Pro:** $1.25 per million input tokens (under 128K), $5 per million output
- At 100K requests/day with a 1K-token prompt: even a 100-token reduction saves $250/month

Tokenization is not just an engineering detail — it is directly tied to the financial unit economics of every LLM-powered product. Production engineers measure and optimize token counts just as they optimize database query times.

Tokens Per Word: Production Rules of Thumb

Rough Estimates for Capacity Planning

- **English prose:** 1 token ≈ 0.75 words; or 100 tokens ≈ 75 words
- **Code (Python/JS):** 1 token ≈ 0.5 words due to indentation and operators
- **Hindi/Arabic:** 1 token ≈ 0.3 words for GPT-4 (cl100k); 3–4x less efficient than English
- **JSON/XML:** 1 token per 3–4 characters; highly structured data is token-expensive
- **Numbers:** Each digit or short number may be 1 token; large numbers split unpredictably

Context Window Budgeting

Allocating Tokens Across the Prompt

- Every LLM has a maximum context length measured in tokens (e.g., GPT-4: 128K, Claude 3: 200K)
- A production prompt has four token consumers: system prompt, conversation history, retrieved context, and response reserve
- Example budget for a RAG chatbot with 32K context: System 500 + History 3000 + RAG Chunks 25000 + Response 3500
- If any budget component overflows, the request fails or content is silently truncated

Context budget management is a core architectural concern in production LLM systems. LangChain, LlamaIndex, and Haystack all provide token-counting middleware to enforce these budgets before each API call.

Context Window Budget Allocation

System Prompt	500 tokens
Conversation History	3,000 tokens
Retrieved Context (RAG Chunks)	25,000 tokens
Response Reserve	3,500 tokens

Total: 32,000 tokens — 32K context window

Each layer is tracked by a token counter middleware. If the total exceeds the model's context limit, the RAG chunks are compressed first (sliding window or summarization), followed by history compression.

Chunking Strategy and Token Alignment

Chunk Size Must Match Tokenizer

- Document chunking splits source text into retrieval units; chunk size is measured in **tokens**, not characters
- A chunk of 512 characters is not 512 tokens — always use the model's tokenizer to count
- Common RAG chunk sizes: 256 tokens (precise), 512 tokens (balanced), 1024 tokens (contextual)
- Overlap (e.g., 50 tokens) between consecutive chunks prevents information loss at boundaries
- LangChain `RecursiveCharacterTextSplitter` includes a tokenizer-based `from_tiktoken_encoder` method

Tokenization in Retrieval and Reranking

Two Tokenization Stages in RAG

- **Embedding stage:** Query and document chunks are tokenized using the *embedding model's* tokenizer (e.g., `text-embedding-3-small` uses cl100k)
- **Generation stage:** Retrieved chunks are assembled into a prompt tokenized by the *generation model's* tokenizer
- Embedding models have their own token limits (typically 8,192 tokens per chunk)
- Rerankers (Cohere Rerank, BGE Reranker) add a third tokenization pass; each has its own limit

A common production bug: developers assume the same tokenizer is used for both embedding and generation. Using a 512-token embedding chunk limit with a model that has 256-token embedding context silently truncates every chunk.

Prompt Compression Techniques

Structural Compression

- Remove markdown formatting from retrieved chunks (saves 5–10%)
- Deduplicate repeated phrases in instructions
- Use abbreviations for repeated terms within a prompt
- Prefer JSON over verbose XML for structured data

Semantic Compression

- LLMLingua (Microsoft): Compresses prompts by 3–5x using a small LM to remove low-importance tokens
- Selective RAG: Retrieve only the top-2 chunks instead of top-5
- Summarize conversation history instead of including verbatim turns

Prompt Caching: Paying Once for Repeated Context

How Prompt Caching Works

- **Anthropic (2024):** Prompt caching stores pre-computed KV-cache for repeated prefix tokens; cached input costs 90% less
- **OpenAI:** Automatic caching for prompts $>$ 1,024 tokens; 50% discount on cached input tokens
- **Use case:** System prompts, large documents, and tool definitions that stay constant across requests
- **Requirement:** The cached prefix must be identical token-for-token; any change invalidates the cache

For a production system with a 10K-token system prompt sent in every request, prompt caching reduces that component's cost by 90%, turning a major cost center into a minor one.

Tokenization Failure Modes in Production

Common Production Bugs Caused by Tokenization

- **Silent truncation:** Input exceeds context limit; tail is silently dropped. Symptoms: incomplete answers, missing data
- **Token split artifacts:** User asks model to count letters in a word; answer is wrong because subword tokens obscure character boundaries
- **Tokenizer mismatch:** System counts tokens with `tiktoken` but calls Claude; token counts differ, context overflow occurs
- **Language inefficiency:** Non-English prompt uses 3x expected tokens; cost and latency spikes unexpectedly

Monitoring Token Usage in Production

LLM Observability for Token Metrics

- **LangSmith (LangChain):** Traces every chain invocation with input/output token counts, latency, and cost
- **LangFuse:** Open-source observability platform; logs token usage, model name, and cost per call
- **Helicone:** Proxy layer between application and OpenAI/Anthropic; aggregates token spend by user, endpoint, and model
- **OpenTelemetry + GenAI Semantic Conventions:** Standardized spans for `llm.token.count.input` and `llm.token.count.output`

Every production LLM system should track token usage as a first-class metric alongside latency and error rate. Token cost dashboards enable anomaly detection when prompt changes cause unexpected token spikes.

Critical: Tokenizer Varies Across Models

Never Assume One Tokenizer Fits All

- **OpenAI GPT-4 / GPT-4o:** cl100k_base or o200k_base (tiktoken)
- **Anthropic Claude:** Custom BPE tokenizer; use `anthropic.count_tokens()` method
- **Meta Llama 3:** tiktoken-style byte-BPE with 128K vocab
- **Google Gemini:** SentencePiece; use `model.count_tokens()` from the Gemini SDK

A prompt counted as 512 tokens by `tiktoken` may be 580 tokens for Claude and 495 for Llama 3. Always use the **target model's own counting method** in production, not a general-purpose counter.

Production Tokenization Checklist

Before Deploying Any LLM Feature

- Use the target model's tokenizer (not a proxy) for all token counting
- Set `max_tokens` and handle truncation explicitly — never rely on silent truncation
- Budget the context window: system + history + retrieval + response reserve
- Measure token fertility on your actual production data, not a general benchmark
- Add token count to every LLM call's telemetry span
- Set cost alerts when daily token spend exceeds thresholds

Production Tokenizer Quick Reference

Model	Tokenizer	Vocab	Count Method
GPT-4 / GPT-4o	tiktoken cl100k	100K	`tiktoken.encoding_for_model()`
GPT-4o (new)	tiktoken o200k	200K	`tiktoken.get_encoding(``o200k_base'`
Claude 3.x / 3.7	Custom BPE	–	`anthropic.count_tokens()`
Llama 3	tiktoken BPE	128K	`AutoTokenizer.from_pretrained()`
Gemini 1.5 / 2.0	SentencePiece	–	`model.count_tokens()`

Key Takeaways

What You Have Learned

- Tokens are the unit of cost, latency, and capacity for every production LLM system
- Context window budgeting (system + history + RAG + response) is a core architectural discipline
- RAG pipelines have two tokenization stages: embedding (retrieval) and generation — each uses its own tokenizer
- Prompt caching (Anthropic, OpenAI) provides 50–90% cost savings for repeated prefixes
- Always use the target model's tokenizer for counting — never assume tokenizer compatibility across providers
- LLM observability tools (LangSmith, LangFuse, Helicone) provide token-level telemetry for production monitoring

Thank You

Tokenization in Production — Module 3.3